

Relatório de Análise de Vulnerabilidades

Projeto: oficina-mvp-backend | Ferramenta-base: SpotBugs + Find Security Bugs

Data da consolidação	23/06/2026
Fonte analisada	Relatório XML/HTML do SpotBugs exportado pelo usuário e revisão complementar do pacote do projeto.
Total de avisos SpotBugs	71
Alta / Média / Baixa	1 / 34 / 36

Resumo executivo. O relatório original do SpotBugs encontrou 71 avisos, mas eles não representam 71 vulnerabilidades exploráveis. A maior parte é composta por endpoints Spring informativos e alertas de exposição de objetos mutáveis. Para entrega, os pontos que merecem prioridade são: CSRF desabilitado, geração previsível de código público de OS, Swagger/OpenAPI exposto, presença de credenciais/secrets padrão no pacote e revisão de autorização por perfil.

1. Métricas consolidadas do SpotBugs

Indicador	Quantidade	Leitura para gestão
Avisos totais	71	Volume bruto do scanner; precisa de triagem.
Alta prioridade	1	Corrigir ou justificar tecnicamente.
Média prioridade	34	Avaliar e priorizar por exposição real.
Baixa prioridade	36	Tratar como hardening/qualidade.
Categoria SECURITY	33	Inclui endpoints Spring e achados reais.
Categoria MALICIOUS_CODE	29	Nome da categoria do SpotBugs; muitos casos são DTOs/coleções mutáveis.

2. Achados priorizados para correção

H-01 - Proteção CSRF desabilitada (Alta)

Evidência	SecurityConfig.java:30 - csrf.disable(); SpotBugs: SPRING_CSRF_PROTECTION_DISABLED / CWE-352.
Impacto	Pode permitir requisições indevidas em aplicações web que usem autenticação baseada em cookie/sessão. Em API stateless com Bearer Token, o risco é menor, mas precisa estar documentado e condicionado ao modelo de autenticação.
Recomendação	Manter CSRF desabilitado somente se a API usar exclusivamente Authorization: Bearer e não cookie de sessão. Caso exista uso por navegador com cookies/credenciais, habilitar CSRF. Documentar a decisão técnica no projeto.
Prioridade sugerida	Alta

M-01 - Código público de ordem de serviço gerado com aleatoriedade previsível (Média)

Evidência	ServiceOrderApplicationService.java:189 - ThreadLocalRandom usado para gerar sufixo da OS; SpotBugs: PREDICTABLE_RANDOM / CWE-330.
Impacto	Como existem endpoints públicos para consulta/aprovação por código da OS, códigos previsíveis podem facilitar enumeração, principalmente se houver tentativa automatizada ou vazamento parcial de dados.
Recomendação	Trocar ThreadLocalRandom por SecureRandom, UUID/ULID ou token público longo e imprevisível. Aplicar rate limit nos endpoints públicos e registrar tentativas inválidas.
Prioridade sugerida	Média

M-02 - Swagger/OpenAPI liberado sem restrição (Média)

Evidência	SecurityConfig.java:36 permite /swagger-ui.html, /swagger-ui/** e /v3/api-docs/**.
Impacto	Em produção, expõe o mapa da API, modelos e endpoints, facilitando reconhecimento por atacante.
Recomendação	Liberar Swagger apenas em ambiente local/desenvolvimento ou proteger com perfil, autenticação ou regra de rede.
Prioridade sugerida	Média

M-03 - Secrets e credenciais padrão presentes no pacote do projeto (Média)

Evidência	Pacote enviado contém .env; application.yml define fallback de JWT secret e senha seed de administrador.
Impacto	Se esses valores forem versionados ou usados fora de desenvolvimento, há risco de acesso indevido, falsificação de token ou login com credenciais previsíveis.
Recomendação	Remover .env do versionamento/pacote de entrega, manter apenas .env.example sem valores reais, rotacionar JWT secret e eliminar senha padrão Admin@123 em ambientes compartilhados.
Prioridade sugerida	Média

M-04 - Endpoints públicos de ordem de serviço precisam de controles adicionais (Média)

Evidência	SecurityConfig.java libera /api/public/**; PublicServiceOrderController expõe consulta e aprovação pública por código e documento.
Impacto	Mesmo com validação por documento, endpoints públicos podem sofrer enumeração, força bruta ou abuso automatizado.
Recomendação	Adicionar rate limiting, auditoria de tentativas, resposta genérica, expiração de código/token e token público forte para aprovação.
Prioridade sugerida	Média

M-05 - Controle de autorização parece amplo (Média)

Evidência	SecurityConfig.java usa anyRequest().authenticated(); não foram identificadas anotações @PreAuthorize/@Secured no pacote analisado.
Impacto	Usuários autenticados podem ter acesso a operações administrativas caso não exista validação de perfil em outra camada.
Recomendação	Definir matriz de permissões e aplicar hasRole/hasAuthority, @PreAuthorize ou validação centralizada por endpoint/serviço.
Prioridade sugerida	Média

B-01 - Header Authorization é entrada não confiável (Baixa)

Evidência	JwtAuthenticationFilter.java:31 - request.getHeader("Authorization"); SpotBugs: SERVLET_HEADER / CWE-807.
Impacto	O alerta é principalmente educativo: headers podem ser manipulados pelo cliente. No código atual, o header é parseado como JWT, o que é esperado.
Recomendação	Não tomar decisões sensíveis apenas com base no header cru. Continuar validando assinatura, expiração e usuário ativo. Padronizar retorno 401 para token inválido.
Prioridade sugerida	Baixa

B-02 - Construtor de JwtService pode lançar exceção (Baixa)

Evidência	JwtService.java:22 - Keys.hmacShaKeyFor(...); SpotBugs: CT_CONSTRUCTOR_THROW.
Impacto	Se o secret for inválido/curto, a aplicação pode falhar ao inicializar. O risco prático é mais de robustez/configuração do que exploração direta.
Recomendação	Validar o JWT secret em configuração/startup com mensagem clara; evitar fallback inseguro e exigir secret forte por variável de ambiente.
Prioridade sugerida	Baixa

3. Achados que parecem ruído ou exigem validação manual

Tipo	Qtd.	Classificação	Como interpretar
SPRING_ENDPOINT	30	Informativo	O SpotBugs apenas indica que métodos de controllers são acessíveis remotamente. Não é vulnerabilidade automática; serve para revisar autenticação/autorização.
EI_EXPOSE_REP / EI_EXPOSE_REP2	29	Ruído provável / qualidade	Muitos achados vêm de records/DTOs e injeção de services em controllers, comuns em Spring. Corrigir coleções mutáveis expostas quando fizer sentido.
THROWS_METHOD_THROWS_CLAUSE_BASIC_EXCEPTION / REC_CATCH_EXCEPTION	3	Qualidade	Refinar tratamento de exceções para evitar captura genérica e mascaramento de erros.
DM_CONVERT_CASE	2	Baixo	Usar Locale.ROOT em normalizações para evitar comportamento dependente do locale da JVM.

4. Distribuição por tipo de alerta

Tipo SpotBugs	Qtd.	Categoria	Prioridade predominante	Observação
SPRING_ENDPOINT	30	SECURITY	Baixa	Informativo; revisar autorização e exposição.
EI_EXPOSE_REP2	16	MALICIOUS_CODE	Média	Normal em injeção de dependência; provável falso positivo em controllers.
EI_EXPOSE_REP	13	MALICIOUS_CODE	Média	Avaliar cópias defensivas em coleções mutáveis.
BX_UNBOXING_IMMEDIATELY_REBOXED	2	PERFORMANCE	Média	Hardening/qualidade de código.
DM_CONVERT_CASE	2	I18N	Baixa	Hardening/qualidade de código.
THROWS_METHOD_THROWS_CLAUSE_BASIC_EXCEPTION	2	BAD_PRACTICE	Baixa	Hardening/qualidade de código.
CT_CONSTRUCTOR_THROW	1	BAD_PRACTICE	Média	Hardening de inicialização do JWT.
PREDICTABLE_RANDOM	1	SECURITY	Média	Trocar por SecureRandom/UUID

				se usado em fluxo público.
REC_CATCH_EXCEPTION	1	STYLE	Baixa	Hardening/qualidade de código.
SERVLET_HEADER	1	SECURITY	Baixa	Validar token; não confiar em header cru.
SPRING_CSRF_PROTECTION_DISABLED	1	SECURITY	Alta	Corrigir ou justificar conforme modelo de autenticação.
UWF_UNWRITTEN_FILED	1	CORRECTNESS	Média	Hardening/qualidade de código.

5. Plano de ação recomendado

- Corrigir ou documentar tecnicamente a decisão de CSRF desabilitado.
- Trocar a geração do código público de OS para valor imprevisível e adicionar rate limit nos endpoints públicos.
- Bloquear Swagger/OpenAPI em produção.
- Remover .env do pacote/repositório e substituir senhas/secrets padrão por variáveis seguras.
- Revisar matriz de autorização por perfil e proteger operações administrativas.
- Reexecutar SpotBugs após correções e complementar com Semgrep/Trivy/ZAP quando Docker/Python estiver disponível.

6. Observações e limitações

- SpotBugs é uma ferramenta de análise estática: ela aponta padrões suspeitos, mas alguns avisos precisam de validação manual.
- Este relatório não substitui teste dinâmico da API. Para cobertura mais completa, recomenda-se complementar com OWASP ZAP, Semgrep e ferramenta de análise de dependências assim que o ambiente permitir.
- Os achados classificados como “ruído provável” não devem ser ignorados automaticamente; eles devem ser registrados como triados ou justificados no backlog técnico.